CSSIMPLER / PERFORMANCE NOTE

What did PR #9 cost?

Paint refreshes during layout transitions · 13 November 2026

Measured in a synthetic CPU test: the fix raised median scene-preparation time by **19%** for full refreshes and **57-59%** for partial refreshes. It also allowed the animation to finish; the old version did not. These are not application frame rates.

Measured cost

Times are microseconds per frame; allocation is requested KiB per frame. Changes use unrounded values.

Refresh	Nodes	Median before → after	P95 before → after	Change median	Alloc. KiB before → after
Idle control	100	211 → 221	276 → 306	+4.8%	1,566 → 1,566
Full paint	100	202 → 239	268 → 300	+18.8%	1,590 → 1,828
Partial paint	100	88 → 137	144 → 192	+56.8%	660 → 987
Idle control	500	991 → 1,003	1,149 → 1,166	+1.2%	6,812 → 6,812
Full paint	500	994 → 1,183	1,220 → 1,339	+19.0%	6,928 → 8,088
Partial paint	500	419 → 666	568 → 850	+58.9%	2,886 → 4,483

What the comparison means

In each active case, the switch was checked seven times per run after enough simulated time to finish: **before: 0/7; after: 7/7**, in all five runs. The earlier version repeatedly restarted the transition, so this is a correctness fix with a real cost, not an equal-work speed comparison. Partial refreshes now clone the destination scene while a transition is active; other transition bookkeeping also changes. Idle allocation counts and bytes were identical; the small idle timing differences may reflect noise.

Method and limits

Apple M1 Pro, 16 GiB, macOS 27.0.1; Rust 1.99.1, release build, one process at a time. Five alternating runs per revision and size; each case warms up for 32 frames then measures 240 frames at simulated 16 ms intervals. The table reports the median of run medians and the median of run P95s. A switch toggles every 32 frames while a status label changes each frame; 100 or 500 additional elements make up the scene. The idle control changes the label without toggling.

`App::frame` includes scene preparation and its returned-scene clone. A counting allocator adds overhead; allocation is cumulative requested bytes, **not** peak memory. No pixels, window presentation, GPU, or real FoldCraft workload were measured. Desktop activity and unlocked clocks limit precision. Findings should guide a realistic follow-up, not predict production FPS.

Before `34472cb` · After `ac4c2ee` · PR #9 · Reproduce: `python3 docs/reports/pr9-paint-refresh-run.py --output /tmp/pr9-results.json`